

Table 1: Pass-sum execution time (s, median per iteration; sum of pass medians, not full executable wall time) and speedup split by pass type. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. When present, the OctTree row includes ColorOctree passes; a separate ColorOctree row reports only those passes. ADT fields = total fields in the selected benchmark ADT (prefer parsed ADT definition; fall back to `@BENCH adt.fields`). SoA bufs = 1 tag buffer plus one buffer for each scalar field, plus one buffer for each non-self packed field annotated Linear, plus recursively counted buffers for each non-self packed field annotated Factored; self-recursive fields add no new buffers. Speedup $>1\times$ means the denominator is faster; **bold** marks $>1.1\times$. Every group contrasts the same two configurations: A_ℓ^{+av} (AoS, loopified, C auto-vectorization enabled) against $S_{\ell bv}^{+av}$ (SoA, loopified, selective buffer sharing, Gibbon SIMD vectorization, C auto-vectorization enabled); Table 3 lists both in full. Nothing in a fold is loopifiable, so neither layout’s fold column gains anything from loopification; measured against the per-program fold tables, both sides of the fold group land within a few percent of that layout’s recursive mutable-cursor column. These two configurations are NOT among the columns of the per-program FOLD tables, which show only the recursive variants, so that table’s $A^{\min}/S^{\min}$ is a comparison between different binaries from this one and may differ from the fold ratio here – in magnitude, and where the two layouts are within a few percent of each other, in direction. The per-program MAP tables do contain both columns. Either way a group here is a pass-SUM, so a program whose passes disagree is reported by whichever dominates the sum. The OctTree row sums its merged programs, and is reported only when every one of them verified in both configurations.

Program	ADT fields	SoA bufs	End-to-end			Fold passes			Map passes		
			A_ℓ^{+av} (s)	$S_{\ell bv}^{+av}$ (s)	$A_\ell^{+av}/S_{\ell bv}^{+av}$	A_ℓ^{+av} (s)	$S_{\ell bv}^{+av}$ (s)	$A_\ell^{+av}/S_{\ell bv}^{+av}$	A_ℓ^{+av} (s)	$S_{\ell bv}^{+av}$ (s)	$A_\ell^{+av}/S_{\ell bv}^{+av}$
OctTree	16	9	0.6426	0.5366	1.20 $\times$	0.3766	0.4849	0.78 $\times$	0.2660	0.0517	5.14 $\times$
Compiler	9	8	0.3273	0.0653	5.01 $\times$	0.1667	0.0434	3.84 $\times$	0.1606	0.0219	7.34 $\times$
DBQuery	15	13	0.2635	0.1157	2.28 $\times$	0.1250	0.0882	1.42 $\times$	0.1385	0.0275	5.03 $\times$
DecisionTree	7	6	0.1354	0.0659	2.06 $\times$	0.1354	0.0659	2.06 $\times$	–	–	–
DecisionTreeClassify	7	6	0.1841	0.1958	0.94 $\times$	0.1841	0.1958	0.94 $\times$	–	–	–
DomTree	15	14	0.2171	0.1392	1.56 $\times$	0.1582	0.0651	2.43 $\times$	0.0589	0.0742	0.79 $\times$
KDTree	17	16	0.1924	0.1997	0.96 $\times$	0.1924	0.1997	0.96 $\times$	–	–	–
LinearListReduction	11	11	0.0320	0.0051	6.33 $\times$	0.0320	0.0051	6.33 $\times$	–	–	–
reduceNestedList	3	3	1.501	0.9668	1.55 $\times$	1.501	0.9668	1.55 $\times$	–	–	–
List	2	2	0.4005	0.3104	1.29 $\times$	0.1541	0.1180	1.31 $\times$	0.2464	0.1924	1.28 $\times$
MonoTree	3	2	0.0555	0.0387	1.43 $\times$	0.0264	0.0232	1.14 $\times$	0.0291	0.0156	1.87 $\times$
ObjectGraph	5	4	0.2602	0.1539	1.69 $\times$	0.1367	0.0827	1.65 $\times$	0.1236	0.0712	1.74 $\times$
TernaryTree	5	3	0.0937	0.0713	1.31 $\times$	0.0293	0.0274	1.07 $\times$	0.0644	0.0439	1.47 $\times$
Trie	10	9	0.2205	0.0761	2.90 $\times$	0.0756	0.0339	2.23 $\times$	0.1450	0.0422	3.44 $\times$
PiecewiseFunctions	8	7	0.3804	0.1491	2.55 $\times$	0.1575	0.0988	1.59 $\times$	0.2229	0.0503	4.43 $\times$
ColorOctree	25	18	0.0903	0.2281	0.40 $\times$	0.0903	0.2281	0.40 $\times$	–	–	–

Table 2: Pass-sum execution time (s, median per iteration; sum of pass medians, not full executable wall time) and speedup split by pass type. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. When present, the OctTree row includes ColorOctree passes; a separate ColorOctree row reports only those passes. ADT fields = total fields in the selected benchmark ADT (prefer parsed ADT definition; fall back to `@BENCH adt.fields`). SoA bufs = 1 tag buffer plus one buffer for each scalar field, plus one buffer for each non-self packed field annotated Linear, plus recursively counted buffers for each non-self packed field annotated Factored; self-recursive fields add no new buffers. Speedup $>1\times$ means the denominator is faster; **bold** marks $>1.1\times$. This table repeats Table 1 with vanilla Gibbon on the AoS side – no optimization enabled, and, as in every column not marked $+av$, the C auto-vectorizer off – so the speedups are what the SoA layout and its optimizations buy over the unoptimized baseline rather than over AoS’s own best configuration. The C auto-vectorizer is held off on both sides of every comparison except the $+av$ columns, which is what keeps it from being credited to whichever Gibbon optimization happens to be measured beside it. Every group contrasts the same two configurations: A_{ri} (AoS, recursive traversal, immutable cursors) against $S_{\ell bv}^{+av}$ (SoA, loopified, selective buffer sharing, Gibbon SIMD vectorization, C auto-vectorization enabled); Table 3 lists both in full. These two configurations are NOT among the columns of the per-program FOLD tables, which show only the recursive variants, so that table’s $A^{\min}/S^{\min}$ is a comparison between different binaries from this one and may differ from the fold ratio here – in magnitude, and where the two layouts are within a few percent of each other, in direction. The per-program MAP tables do contain both columns. Either way a group here is a pass-SUM, so a program whose passes disagree is reported by whichever dominates the sum. The OctTree row sums its merged programs, and is reported only when every one of them verified in both configurations.

Program	ADT fields	SoA bufs	End-to-end			Fold passes			Map passes		
			A_{ri} (s)	$S_{\ell bv}^{+av}$ (s)	$A_{ri}/S_{\ell bv}^{+av}$	A_{ri} (s)	$S_{\ell bv}^{+av}$ (s)	$A_{ri}/S_{\ell bv}^{+av}$	A_{ri} (s)	$S_{\ell bv}^{+av}$ (s)	$A_{ri}/S_{\ell bv}^{+av}$
OctTree	16	9	0.7620	0.5366	1.42 $\times$	0.4519	0.4849	0.93 $\times$	0.3101	0.0517	6.00 $\times$
Compiler	9	8	1.238	0.0653	18.95 $\times$	0.7776	0.0434	17.91 $\times$	0.4600	0.0219	21.02 $\times$
DBQuery	15	13	0.3482	0.1157	3.01 $\times$	0.1790	0.0882	2.03 $\times$	0.1692	0.0275	6.15 $\times$
DecisionTree	7	6	0.3047	0.0659	4.62 $\times$	0.3047	0.0659	4.62 $\times$	–	–	–
DecisionTreeClassify	7	6	0.1925	0.1958	0.98 $\times$	0.1925	0.1958	0.98 $\times$	–	–	–
DomTree	15	14	0.3659	0.1392	2.63 $\times$	0.3012	0.0651	4.63 $\times$	0.0647	0.0742	0.87 $\times$
KDTree	17	16	0.2623	0.1997	1.31 $\times$	0.2623	0.1997	1.31 $\times$	–	–	–
LinearListReduction	11	11	0.1456	0.0051	28.79 $\times$	0.1456	0.0051	28.79 $\times$	–	–	–
reduceNestedList	3	3	17.61	0.9668	18.22 $\times$	17.61	0.9668	18.22 $\times$	–	–	–
List	2	2	–	0.3104	–	–	0.1180	–	–	0.1924	–
MonoTree	3	2	0.1351	0.0387	3.49 $\times$	0.0820	0.0232	3.54 $\times$	0.0531	0.0156	3.41 $\times$
ObjectGraph	5	4	0.4842	0.1539	3.15 $\times$	0.3156	0.0827	3.82 $\times$	0.1686	0.0712	2.37 $\times$
TernaryTree	5	3	0.1359	0.0713	1.91 $\times$	0.0516	0.0274	1.88 $\times$	0.0843	0.0439	1.92 $\times$
Trie	10	9	0.3131	0.0761	4.11 $\times$	0.1497	0.0339	4.41 $\times$	0.1634	0.0422	3.87 $\times$
PiecewiseFunctions	8	7	0.5775	0.1491	3.87 $\times$	0.3163	0.0988	3.20 $\times$	0.2612	0.0503	5.19 $\times$
ColorOctree	25	18	0.1139	0.2281	0.50 $\times$	0.1139	0.2281	0.50 $\times$	–	–	–

Table 3: Configuration key for the per-program fold and map tables. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. Base letter: A = array-of-structs (AoS), S = struct-of-arrays (SoA). Subscripts name the Gibbon-side code generation (r recursive traversal, i immutable cursors, m mutable cursors, ℓ loopified, b selective buffer sharing, v Gibbon SIMD vectorization). Superscripts name a C-compiler knob that DIFFERS from the default, always with a leading minus: $-t$ compiles with C tail-call optimization disabled and $-av$ with the C auto-vectorizer disabled. The auto-vectorizer is ON in every column that does not say otherwise, because that is what an ordinary build does, so no comparison between two columns can confuse it with the optimization being measured. Columns marked $-av$ appear only when the auto-vectorizer ablations were requested, and are there to be read against their unmarked counterpart. The recursive configurations of each layout carry the fold passes; map tables additionally report the loopified configurations.

Symbol	Configuration
A_{ri}^{-t}	AoS, recursive traversal, immutable cursors, C tail-call optimization disabled
A_{ri}	AoS, recursive traversal, immutable cursors
A_{rm}^{-t}	AoS, recursive traversal, mutable cursors, C tail-call optimization disabled
A_{rm}^{-av}	AoS, recursive traversal, mutable cursors, C auto-vectorization disabled
A_{rm}^{+av}	AoS, recursive traversal, mutable cursors, C auto-vectorization enabled
A_{ℓ}^{-av}	AoS, loopified, C auto-vectorization disabled
A_{ℓ}^{+av}	AoS, loopified, C auto-vectorization enabled
S_{ri}^{-t}	SoA, recursive traversal, immutable cursors, C tail-call optimization disabled
S_{ri}	SoA, recursive traversal, immutable cursors
S_{rm}^{-t}	SoA, recursive traversal, mutable cursors, C tail-call optimization disabled
S_{rm}^{-av}	SoA, recursive traversal, mutable cursors, C auto-vectorization disabled
S_{rm}^{+av}	SoA, recursive traversal, mutable cursors, C auto-vectorization enabled
S_{ℓ}^{-av}	SoA, loopified, no selective buffer sharing, C auto-vectorization disabled
S_{ℓ}^{+av}	SoA, loopified, no selective buffer sharing, C auto-vectorization enabled
$S_{\ell b}^{-av}$	SoA, loopified, selective buffer sharing, C auto-vectorization disabled
$S_{\ell b}^{+av}$	SoA, loopified, selective buffer sharing, C auto-vectorization enabled
$S_{\ell bv}^{-av}$	SoA, loopified, selective buffer sharing, Gibbon SIMD vectorization, C auto-vectorization disabled
$S_{\ell bv}^{+av}$	SoA, loopified, selective buffer sharing, Gibbon SIMD vectorization, C auto-vectorization enabled

Table 4: Key for the per-pass delta tables. Each column is $(baseline - feature\ enabled) / feature\ enabled$, expressed as a percentage, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Normalizing to the FEATURE rather than the baseline is deliberate – against the baseline the scale saturates, with a $5\times$ win reading 80% and an $8\times$ win 87.5%, so large improvements compress into a narrow band and stop being distinguishable. Superscript names the layout (A = array-of-structs, S = struct-of-arrays); subscript names the feature. The two |-subscripted columns ask whether one vectorizer helps or hinders the other, which is why these are signed differences rather than magnitudes. A subscript after a | names what the feature was added ON TOP OF, so $\Delta_{av|\ell}^A$ is the C auto-vectorizer applied to loopified code. Δ_{ℓ}^S additionally turns on scalar-count footers, which SoA loopification cannot run without; measured on their own they cost between -1 and $+3$ percent, so the column is loopification for practical purposes. Each column’s formula below names the exact pair it subtracts, which is also where to read off whether the C auto-vectorizer was on or off for it. Table 3 defines the configuration symbols themselves.

Column	Meaning: (baseline – feature) / feature
Δ_m^A	What mutable cursors alone bought vanilla Gibbon. BOTH sides have the C tail-call optimization disabled, so this isolates mutability: measured against A_{ri} (tail calls ENABLED) it would report mutability plus whatever tail calls contributed, and mutability is precisely what puts the traversal in tail position for them to work on: $(A_{ri}^{-t} - A_{rm}^{-t})/A_{rm}^{-t}$
Δ_t^A	What the C tail-call optimization bought AoS mutable. Mutable cursors are what leave the traversal in tail position, so this optimization has something to work on only once they are in use: $(A_{rm}^{-t} - A_{rm}^{+av})/A_{rm}^{+av}$
Δ_m^S	What mutable cursors alone bought SoA; as in AoS, both sides have the C tail-call optimization disabled so the two effects are not conflated: $(S_{ri}^{-t} - S_{rm}^{-t})/S_{rm}^{-t}$
Δ_t^S	What the C tail-call optimization bought SoA mutable; as in AoS, it is mutability that leaves the traversal in tail position: $(S_{rm}^{-t} - S_{rm}^{+av})/S_{rm}^{+av}$
$\Delta_{av rm}^A$	What the C auto-vectorizer added to AoS, recursive traversal, mutable cursors: $(A_{rm}^{-av} - A_{rm}^{+av})/A_{rm}^{+av}$
Δ_{ℓ}^A	What loopification gained over recursion in AoS mutable, with the C auto-vectorizer off on BOTH sides so that it measures loopification and nothing else: $(A_{rm}^{-av} - A_{\ell}^{-av})/A_{\ell}^{-av}$
$\Delta_{av \ell}^A$	What the C auto-vectorizer added to AoS, loopified: $(A_{\ell}^{-av} - A_{\ell}^{+av})/A_{\ell}^{+av}$
$\Delta_{av rm}^S$	What the C auto-vectorizer added to SoA, recursive traversal, mutable cursors: $(S_{rm}^{-av} - S_{rm}^{+av})/S_{rm}^{+av}$
Δ_{ℓ}^S	What loopification gained over recursion in SoA mutable, with the C auto-vectorizer off on BOTH sides; neither side has selective buffer sharing, so this is loopification alone: $(S_{rm}^{-av} - S_{\ell}^{-av})/S_{\ell}^{-av}$
$\Delta_{av \ell}^S$	What the C auto-vectorizer added to SoA, loopified, no selective buffer sharing: $(S_{\ell}^{-av} - S_{\ell}^{+av})/S_{\ell}^{+av}$
Δ_b^S	What selective buffer sharing added over SoA loopified, with the C auto-vectorizer off on both sides: $(S_{\ell}^{-av} - S_{\ell b}^{-av})/S_{\ell b}^{-av}$
$\Delta_{av \ell b}^S$	What the C auto-vectorizer added to SoA, loopified, selective buffer sharing: $(S_{\ell b}^{-av} - S_{\ell b}^{+av})/S_{\ell b}^{+av}$
Δ_v^S	What Gibbon’s SIMD vectorization added over SoA loopified + selective buffer sharing, with the C auto-vectorizer off on BOTH sides. Measured with it ON, this column reports only what is left after the backend has already vectorized the loop, which is nothing — the $\Delta_{av \ell b}^S$ column is where the backend’s own contribution is reported: $(S_{\ell b}^{-av} - S_{\ell bv}^{-av})/S_{\ell bv}^{-av}$
$\Delta_{av \ell bv}^S$	What the C auto-vectorizer added to SoA, loopified, selective buffer sharing, Gibbon SIMD vectorization: $(S_{\ell bv}^{-av} - S_{\ell bv}^{+av})/S_{\ell bv}^{+av}$

Table 5: Per-pass fold performance for **Compiler**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
verifyIR	9/9	0%	0.08209	0.08413	0.06326	0.01352	‡	‡	0.04563	0.003561	3.80 $\times$
instCountPass	2/9	78%	0.08175	0.08596	0.06361	0.01338	‡	‡	0.04413	0.001964	6.81 $\times$
blockCountPass	2/9	78%	0.08593	0.08492	0.06323	0.01378	‡	‡	0.04376	0.001958	7.04 $\times$
memoryOpStatsPass	3/9	67%	0.08617	0.08477	0.06542	0.01409	‡	‡	0.0482	0.003889	3.62 $\times$
castInstCountPass	3/9	67%	0.08235	0.08483	0.06423	0.01354	‡	‡	0.04615	0.003439	3.94 $\times$
branchStatsPass	2/9	78%	0.08196	0.08461	0.06436	0.01353	‡	‡	0.04618	0.003494	3.87 $\times$
latencyModelPass	3/9	67%	0.0825	0.08497	0.06344	0.01405	‡	‡	0.04606	0.003024	4.65 $\times$
has cycle	4/9	56%	0.09628	0.09826	0.0642	0.05881	‡	‡	0.04636	0.01784	3.30 $\times$
throughputModelPass	3/9	67%	0.0824	0.08517	0.06349	0.01361	‡	‡	0.04547	0.003091	4.40 $\times$

Table 6: What each optimization bought, per fold pass, for **Compiler** (percent; companion to Table 5). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
verifyIR	+29.76%	+367.8%	-	+1181%
instCountPass	+28.51%	+375.4%	-	+2147%
blockCountPass	+35.9%	+359%	-	+2135%
memoryOpStatsPass	+31.72%	+364.3%	-	+1139%
castInstCountPass	+28.21%	+374.3%	-	+1242%
branchStatsPass	+27.33%	+375.7%	-	+1222%
latencyModelPass	+30.05%	+351.6%	-	+1423%
has cycle	+49.96%	+9.178%	-	+159.8%
throughputModelPass	+29.77%	+366.7%	-	+1371%

Table 7: Per-pass map performance for **Compiler**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so “fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS								SoA										$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$	$S_{\ell bv}^{+av}$	
targetReturnPass	7/8	88%	0.2116	0.2275	0.2248	0.08627	0.08443	0.08185	0.08101	‡	‡	0.2182	0.1058	0.09886	0.1067	0.1077	0.01163	0.01112	0.01138	0.01097	7.39 $\times$
stripSideEffectsPass	7/8	88%	0.2163	0.2324	0.1879	0.08103	0.08184	0.07962	0.07958	‡	‡	0.2137	0.1008	0.09959	0.1057	0.1055	0.01089	0.01091	0.0105	0.01092	7.58 $\times$

Table 8: What each optimization bought, per map pass, for **Compiler** (percent; companion to Table 7). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
targetReturnPass	-5.863%	+166.2%	+2.178%	+5.4%	+1.029%	-	+120.7%	+7.01%	-0.8854%	-0.8841%	+817.4%	+4.65%	+2.259%	+3.729%
stripSideEffectsPass	+15.15%	+129.5%	-0.997%	+1.771%	+0.04272%	-	+114.6%	+1.182%	-4.642%	+0.1365%	+870.7%	-0.2291%	+3.656%	-3.793%

Table 9: Per-pass fold performance for **DBQuery**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
sumCost	6/15	60%	0.02898	0.02851	0.02014	0.01977	0.0947	0.09395	0.01699	0.01474	1.34 $\times$
sumRows	6/15	60%	0.03071	0.03053	0.02348	0.02633	0.09514	0.09335	0.02022	0.02119	1.16 $\times$
countJoins	3/15	80%	0.02942	0.02939	0.02151	0.02011	0.09148	0.09139	0.01399	0.01122	1.79 $\times$
sumMemory	6/15	60%	0.02884	0.02876	0.02007	0.01949	0.09508	0.09486	0.01689	0.0146	1.34 $\times$
hashJoinPressure	5/15	67%	0.0333	0.03295	0.02094	0.02026	0.09145	0.09201	0.01489	0.01305	1.55 $\times$
filterSelectivitySkew	5/15	67%	0.02917	0.02883	0.02099	0.01909	0.09355	0.09548	0.01613	0.01274	1.50 $\times$

Table 10: What each optimization bought, per fold pass, for **DBQuery** (percent; companion to Table 9). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
sumCost	+43.9%	+1.897%	+457.6%	+15.19%
sumRows	+30.81%	-10.83%	+370.5%	-4.577%
countJoins	+36.74%	+6.977%	+554%	+24.65%
sumMemory	+43.71%	+2.945%	+463%	+15.7%
hashJoinPressure	+59.01%	+3.371%	+514.4%	+14.1%
filterSelectivitySkew	+38.97%	+9.982%	+480%	+26.6%

Table 11: Per-pass map performance for **DBQuery**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so “fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS										SoA										
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$	$S_{\ell bv}^{+av}$	$A^{\min}/S^{\min}$
scaleCosts	10/13	77%	0.08471	0.08557	0.07804	0.07392	0.0715	0.06818	0.0701	0.3266	0.3306	0.1004	0.09181	0.09258	0.08135	0.08323	0.0191	0.01866	0.01878	0.0189	3.65 ×
clearQueryFlags	12/13	92%	0.08363	0.08367	0.07785	0.074	0.07035	0.0677	0.06837	0.3292	0.328	0.1001	0.08992	0.09126	0.08121	0.08171	0.008748	0.008408	0.008519	0.008611	8.05 ×

Table 12: What each optimization bought, per map pass, for **DBQuery** (percent; companion to Table 11). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_{ℓ}^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_{ℓ}^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
scaleCosts	+8.554%	+9.135%	+3.383%	+8.42%	-2.737%	+225.2%	+8.463%	-0.8349%	+12.86%	-2.264%	+326%	+2.337%	+1.683%	-0.6191%
clearQueryFlags	+7.42%	+10.67%	+5.194%	+9.299%	-0.977%	+228.8%	+9.716%	-1.472%	+10.72%	-0.6034%	+828.4%	+4.044%	+2.688%	-1.068%

Table 13: Per-pass fold performance for **DecisionTree**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				$A^{\min}/S^{\min}$
			A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
countNodes	2/7	71%	0.03447	0.03446	0.01712	0.01485	0.1403	0.1402	0.01053	0.005959	2.49 $\times$
countLeaves	2/7	71%	0.03427	0.03415	0.017	0.01475	0.1404	0.1396	0.01106	0.005702	2.59 $\times$
treeDepth	2/7	71%	0.03394	0.03389	0.01671	0.01778	0.14	0.1391	0.01417	0.006628	2.52 $\times$
sumImpurity	3/7	57%	0.03389	0.03381	0.01546	0.01425	0.1446	0.1431	0.01382	0.007762	1.84 $\times$
sumSamples	3/7	57%	0.03298	0.03295	0.01735	0.01425	0.1392	0.1388	0.01037	0.00782	1.82 $\times$
countFeatureUses	3/7	57%	0.03458	0.03441	0.01579	0.01431	0.1462	0.1458	0.01415	0.007826	1.83 $\times$
countSmallLeaves	3/7	57%	0.03352	0.03332	0.01662	0.01385	0.1438	0.1449	0.01027	0.007916	1.75 $\times$
inferenceCost	2/7	71%	0.03397	0.03386	0.01801	0.01689	0.14	0.1398	0.01393	0.007693	2.20 $\times$
sumPathLengths	3/7	57%	0.03392	0.03381	0.01716	0.0148	0.1424	0.1416	0.01385	0.00831	1.78 $\times$

Table 14: What each optimization bought, per fold pass, for **DecisionTree** (percent; companion to Table 13). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass			AoS		SoA	
			Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
countNodes			+101.4%	+15.3%	+1232%	+76.76%
countLeaves			+101.6%	+15.26%	+1169%	+93.95%
treeDepth			+103.1%	-6.045%	+887.8%	+113.8%
sumImpurity			+119.2%	+8.498%	+946.7%	+78.01%
sumSamples			+90.07%	+21.73%	+1243%	+32.58%
countFeatureUses			+119%	+10.3%	+933.6%	+80.79%
countSmallLeaves			+101.7%	+19.99%	+1300%	+29.74%
inferenceCost			+88.63%	+6.619%	+905.2%	+81.06%
sumPathLengths			+97.62%	+15.98%	+928%	+66.73%

Table 15: Per-pass fold performance for **DecisionTreeClassify**. **Measured with random-access nodes enabled** (no `--no-ran`), unlike every other benchmark in this campaign: without them this program measures subtree-skipping rather than the traversal it names. See `RAN_ENABLED_PROGRAMS` for the justification and its re-qualification against the oracle. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				$A^{\min}/S^{\min}$
			A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
classify Depth	4/7	43%	0.04202	0.03871	0.04247	0.03717	0.06322	0.05939	0.04977	0.0405	0.92 $\times$
classify Batch	5/7	29%	0.1662	0.1538	0.1699	0.1467	0.2513	0.2353	0.1969	0.1592	0.92 $\times$

Table 16: What each optimization bought, per fold pass, for **DecisionTreeClassify** (percent; companion to Table 15). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass			AoS		SoA	
			Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
classify Depth			-1.062%	+14.27%	+27.04%	+22.88%
classify Batch			-2.144%	+15.8%	+27.68%	+23.66%

Table 17: Per-pass fold performance for **DomTree**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
SumArea	6/15	60%	0.0715	0.07379	0.04102	0.03989	0.2832	0.2834	0.01972	0.01932	2.07×
find max Bottom	5/15	67%	0.07084	0.07807	0.04126	0.0401	0.2241	0.2231	0.02307	0.01851	2.17×
count styled	3/15	80%	0.071	0.07462	0.03997	0.03926	0.2079	0.2082	0.01674	0.01268	3.10×
sumTextWidth	3/15	80%	0.07098	0.07473	0.03993	0.03881	0.2203	0.22	0.01549	0.01224	3.17×

Table 18: What each optimization bought, per fold pass, for **DomTree** (percent; companion to Table 17). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass			AoS		SoA	
			Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
SumArea			+74.32%	+2.813%	+1336%	+2.076%
find max Bottom			+71.69%	+2.883%	+871.1%	+24.65%
count styled			+77.64%	+1.806%	+1142%	+32.07%
sumTextWidth			+77.79%	+2.878%	+1322%	+26.58%

Table 19: Per-pass map performance for **DomTree**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS								SoA										$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	A_{rm}^{+v}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	
computeWidths	13/14	93%	0.03309	0.03312	0.03144	0.032	0.03194	0.03175	0.03121	0.1115	0.1111	0.05774	0.06377	0.06006	0.06833	0.06313	0.06523	0.06252	0.06502	0.0625	$0.54\times$
scaleLayout	8/14	57%	0.03088	0.03161	0.03077	0.0307	0.03214	0.02823	0.02765	0.09086	0.09108	0.0385	0.04054	0.03884	0.03679	0.03661	0.0125	0.01159	0.01249	0.01166	$2.38\times$

Table 20: What each optimization bought, per map pass, for **DomTree** (percent; companion to Table 19). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass			AoS					SoA								
			Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_{ℓ}^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_{ℓ}^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell b v}^S$
computeWidths	+5.271%	-1.569%	+0.2004%	+0.8032%	+1.724%	+93.16%	-3.871%	+6.175%	-6.671%	+8.241%	+4.757%	+4.325%	+0.3091%	+4.043%		
scaleLayout	+0.3445%	-4.262%	-4.486%	+8.754%	+2.098%	+136%	-0.8805%	+4.385%	+10.2%	+0.4917%	+194.4%	+7.789%	+0.08009%	+7.084%		

Table 21: Per-pass fold performance for **KDTree**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
Find nearest Neighbour	13/17	24%	0.0427	0.04195	0.02974	0.03068	0.1058	0.1076	0.02465	0.02424	1.23 $\times$
countInRange tight_box	13/17	24%	0.04422	0.04292	0.02908	0.03027	0.08652	0.08817	0.02371	0.02394	1.23 $\times$
sumMassInRange	14/17	18%	0.0438	0.04262	0.029	0.02931	0.07961	0.07974	0.02528	0.02274	1.28 $\times$
twoPointCorrelation bin_8_16	11/17	35%	0.04415	0.04342	0.03195	0.03198	0.1072	0.1091	0.02972	0.02931	1.09 $\times$
pointCloudNeighborhood	11/17	35%	0.04285	0.04208	0.02865	0.02792	0.1058	0.1079	0.02193	0.02177	1.28 $\times$
photonMappingBenchmark	12/17	29%	0.05029	0.04931	0.04338	0.0428	0.08885	0.08887	0.04379	0.04346	0.98 $\times$

Table 22: What each optimization bought, per fold pass, for **KDTree** (percent; companion to Table 21). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
Find nearest Neighbour	+43.6%	-3.061%	+329.1%	+1.671%
countInRange tight_box	+52.07%	-3.925%	+264.9%	-0.9482%
sumMassInRange	+51.03%	-1.047%	+214.9%	+11.18%
twoPointCorrelation bin_8_16	+38.19%	-0.09381%	+260.8%	+1.385%
pointCloudNeighborhood	+49.53%	+2.636%	+382.7%	+0.7119%
photonMappingBenchmark	+15.93%	+1.353%	+102.9%	+0.7478%

Table 23: Per-pass fold performance for **LinearListReduction**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (-mavx2); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS						SoA				
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	$A^{\min}/S^{\min}$
reduction	2/11	82%	0.137	0.1456	0.07507	0.03212	$\frac{1}{2}$	$\frac{1}{2}$	0.07145	0.005241	6.13×

Table 24: What each optimization bought, per fold pass, for **LinearListReduction** (percent; companion to Table 23). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
reduction	+82.51%	+133.7%	-	+1263%

Table 25: Per-pass fold performance for **reduceNestedList**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (-mavx2); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS						SoA				$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
reduction	2/3	33%	17.69	17.61	10.01	1.768	17.71	17.73	9.539	0.9783	1.81 $\times$

Table 26: What each optimization bought, per fold pass, for **reduceNestedList** (percent; companion to Table 25). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
reduction	+76.61%	+466.4%	+85.62%	+875%

Table 27: Per-pass fold performance for **List**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (-mavx2); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS						SoA					$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}		
length List	1/2	50%	‡	‡	‡	0.051	‡	‡	‡	0.02256	2.26×	
sumList	2/2	0%	‡	‡	‡	0.05136	‡	‡	‡	0.06123	0.84×	
sumList tail recursive	2/2	0%	‡	‡	‡	0.05168	‡	‡	‡	0.06148	0.84×	

Table 28: What each optimization bought, per fold pass, for **List** (percent; companion to Table 27). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘—’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
length List	—	—	—	—
sumList	—	—	—	—
sumList tail recursive	—	—	—	—

Table 29: Per-pass map performance for **List**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS										SoA										$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	
add1 List	1/2	50%	‡	‡	‡	0.2763	0.2707	0.246	0.2464	‡	‡	‡	0.3022	0.3564	0.2585	0.2366	0.2063	0.1938	0.1926	0.1924	1.28 ×

Table 30: What each optimization bought, per map pass, for **List** (percent; companion to Table 29). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘—’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA									
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av lb}^S$	Δ_v^S	$\Delta_{av lbv}^S$	
add1 List	−	−	+2.05%	+12.31%	−0.168%	−	−	−15.19%	+16.89%	+9.294%	+25.3%	+6.483%	+7.117%	+0.119%	

Table 31: Per-pass fold performance for **MonoTree**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
sumTree	3/3	0%	0.04055	0.04086	0.0193	0.01449	0.0545	0.0546	0.01568	0.01185	1.22×
sumTree TailRec	3/3	0%	0.04087	0.04112	0.01771	0.01291	0.05285	0.05286	0.01858	0.01142	1.13×

Table 32: What each optimization bought, per fold pass, for **MonoTree** (percent; companion to Table 31). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘—’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
sumTree	+110.1%	+33.22%	+247.5%	+32.33%
sumTree TailRec	+130.8%	+37.16%	+184.5%	+62.62%

Table 33: Per-pass map performance for **MonoTree**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS									SoA											$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	
add1Tree	1/2	50%	0.05489	0.05309	0.04444	0.04187	0.04252	0.03045	0.02909	0.109	0.1091	0.0424	0.04466	0.04496	0.02279	0.02047	0.01622	0.01565	0.01555	0.01557	1.87 $\times$

Table 34: What each optimization bought, per map pass, for **MonoTree** (percent; companion to Table 33). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS						SoA							
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_{ℓ}^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_{ℓ}^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell b v}^S$
add1Tree	+23.53%	+4.496%	-1.533%	+37.5%	+4.696%	+157%	-5.702%	-0.6672%	+95.94%	+11.34%	+40.56%	+3.623%	+4.289%	-0.1028%

Table 35: Per-pass fold performance for **ObjectGraph**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass				AoS				SoA				$A^{\min}/S^{\min}$
	Uses	Dead%		A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
totalHeapSize	3/5	40%		0.04424	0.04443	0.02106	0.02077	0.07194	0.07135	0.01652	0.01077	1.93 $\times$
countMarked	3/5	40%		0.0454	0.04597	0.01902	0.01839	0.08005	0.0793	0.01729	0.01082	1.70 $\times$
countLargeItems	3/5	40%		0.04521	0.0458	0.02176	0.01933	0.08075	0.0801	0.01599	0.01099	1.76 $\times$
liveBytes	4/5	20%		0.04417	0.04464	0.0223	0.01948	0.08051	0.08006	0.02105	0.01321	1.47 $\times$
deadBytes	4/5	20%		0.04492	0.04549	0.02207	0.01837	0.08048	0.08003	0.02223	0.01317	1.39 $\times$
countSurvivors	4/5	20%		0.04433	0.04473	0.02305	0.0202	0.0794	0.0801	0.02244	0.01382	1.46 $\times$
sumObjIds	3/5	40%		0.04413	0.04458	0.02032	0.01831	0.08006	0.08011	0.0192	0.01058	1.73 $\times$

Table 36: What each optimization bought, per fold pass, for **ObjectGraph** (percent; companion to Table 35). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
totalHeapSize	+110.1%	+1.391%	+335.5%	+53.37%
countMarked	+138.8%	+3.376%	+362.9%	+59.81%
countLargeItems	+107.8%	+12.53%	+404.9%	+45.53%
liveBytes	+98.09%	+14.46%	+282.5%	+59.35%
deadBytes	+103.5%	+20.17%	+262.1%	+68.74%
countSurvivors	+92.34%	+14.13%	+253.9%	+62.39%
sumObjIds	+117.2%	+11.01%	+316.9%	+81.55%

Table 37: Per-pass map performance for **ObjectGraph**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (*b*) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS										SoA										$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	
sweepUnmarked	2/4	50%	0.08498	0.08486	0.07294	0.07395	0.07337	0.06175	0.06185	0.1674	0.1697	0.09546	0.07927	0.09534	0.05894	0.0546	0.03747	0.03751	0.03308	0.03341	1.87 $\times$
touchHotObjects	2/4	50%	0.08364	0.08371	0.07602	0.07388	0.07492	0.06204	0.0617	0.1731	0.174	0.09613	0.07994	0.09797	0.06171	0.05678	0.04391	0.03746	0.03972	0.0378	1.65 $\times$

Table 38: What each optimization bought, per map pass, for **ObjectGraph** (percent; companion to Table 37). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA									
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell b v}^S$	
sweepUnmarked	+16.5%	-0.5833%	+0.7891%	+19.76%	-0.1633%	+75.32%	+0.1217%	-16.86%	+34.49%	+7.949%	+57.31%	-0.12%	+13.27%	-0.9908%	
touchHotObjects	+10.02%	+1.472%	-1.383%	+19.08%	+0.5494%	+80.01%	-1.874%	-18.4%	+29.54%	+8.69%	+40.55%	+17.2%	+10.54%	+5.076%	

Table 39: Per-pass fold performance for **OctTree.sumMass**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS					SoA					$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
sumMass	10/16	38%	0.04443	0.04454	0.03392	0.03537	‡	‡	0.0274	0.02685	1.26 $\times$

Table 40: What each optimization bought, per fold pass, for **OctTree.sumMass** (percent; companion to Table 39). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
sumMass	+30.99%	-4.113%	-	+2.045%

Table 41: Per-pass fold performance for **OctTree.sumEnergy**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS					SoA					$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
sumEnergy	12/16	25%	0.0495	0.04901	0.03672	0.03796	‡	‡	0.03364	0.03051	1.20 $\times$

Table 42: What each optimization bought, per fold pass, for **OctTree.sumEnergy** (percent; companion to Table 41). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
sumEnergy	+34.8%	-3.259%	-	+10.28%

Table 43: Per-pass fold performance for `OctTree.countActive`. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
countActive	10/16	38%	0.04267	0.0424	0.0343	0.03495	‡	‡	0.02682	0.02785	1.28 $\times$

Table 44: What each optimization bought, per fold pass, for `OctTree.countActive` (percent; companion to Table 43). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
countActive	+24.4%	-1.854%	-	-3.702%

Table 45: Per-pass fold performance for `OctTree.countParticles`. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
countParticles	8/16	50%	0.04248	0.04247	0.03368	0.03465	‡	‡	0.02733	0.02735	1.23 $\times$

Table 46: What each optimization bought, per fold pass, for `OctTree.countParticles` (percent; companion to Table 45). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
countParticles	+26.11%	-2.779%	-	-0.06582%

Table 47: Per-pass fold performance for `OctTree.barnesHutPotential`. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
barnesHutPotential	11/16	31%	0.05954	0.05934	0.04338	0.04368	‡	‡	0.04233	0.04584	1.02 $\times$

Table 48: What each optimization bought, per fold pass, for `OctTree.barnesHutPotential` (percent; companion to Table 47). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
barnesHutPotential	+37.25%	-0.6959%	-	-7.672%

Table 49: Per-pass fold performance for `OctTree.fmmPotential`. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
fmmPotential	12/16	25%	0.1042	0.1002	0.09979	0.09957	‡	‡	0.09727	0.09804	1.02×

Table 50: What each optimization bought, per fold pass, for `OctTree.fmmPotential` (percent; companion to Table 49). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
fmmPotential	+4.378%	+0.231%	-	-0.7813%

Table 51: Per-pass map performance for `OctTree.scaleEnergy`. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (*b*) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS										SoA										$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$	$S_{\ell bv}^{+av}$	
scaleEnergy	7/9	78%	0.153	0.1577	0.1526	0.1616	0.1751	0.1397	0.1368	$\ddagger$	$\ddagger$	0.196	0.1907	0.19	0.1522	0.146	0.04641	0.04593	0.04566	0.0457	$3.00\times$

Table 52: What each optimization bought, per map pass, for `OctTree.scaleEnergy` (percent; companion to Table 51). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS						SoA									
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_{ℓ}^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_{ℓ}^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$		
scaleEnergy	+0.2202%	-12.85%	-7.733%	+15.7%	+2.06%	-	+3.131%	+0.3831%	+25.28%	+4.252%	+228%	+1.058%	+1.645%	-0.0919%		

Table 53: Per-pass map performance for `OctTree.clearFlags`. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (*b*) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS										SoA										$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$	$S_{\ell bv}^{+av}$	
clearFlags	8/9	89%	0.1525	0.1524	0.1518	0.1567	0.1572	0.1298	0.1292	‡	‡	0.1874	0.1845	0.1828	0.1415	0.1337	0.006101	0.006059	0.006013	0.005999	21.53×

Table 54: What each optimization bought, per map pass, for `OctTree.clearFlags` (percent; companion to Table 53). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA									
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$	
clearFlags	+0.448%	-3.454%	-0.3047%	+20.74%	+0.4924%	-	+2.529%	+0.9745%	+30.39%	+5.893%	+2220%	+0.6932%	+1.463%	+0.2334%	

Table 55: Per-pass fold performance for **TernaryTree**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
sum tree	5/5	0%	0.05176	0.05161	0.02853	0.02862	0.2814	0.2715	0.02939	0.02961	0.97×

Table 56: What each optimization bought, per fold pass, for **TernaryTree** (percent; companion to Table 55). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
sum tree	+81.41%	-0.3144%	+857.5%	-0.7632%

Table 57: Per-pass map performance for **TernaryTree**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so “fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS										SoA										$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	S_{b}^{-av}	S_{b}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	
add 1 tree	1/3	33%	0.09886	0.0843	0.07444	0.08073	0.08098	0.05998	0.0644	0.3811	0.3777	0.08228	0.08172	0.08074	0.05319	0.05009	0.04478	0.04276	0.04616	0.04391	1.40 ×

Table 58: What each optimization bought, per map pass, for **TernaryTree** (percent; companion to Table 57). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA									
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$	
add 1 tree	+32.81%	−8.085%	−0.3161%	+34.58%	−6.864%	+363.2%	+1.901%	+1.208%	+53.64%	+6.183%	+18.77%	+4.726%	−2.987%	+5.129%	

Table 59: Per-pass fold performance for **Trië**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS						SoA				$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
sumPrefixFreq	3/10	70%	0.02926	0.0297	0.01469	0.0148	0.1044	0.104	0.008463	0.006003	2.45 ×
countTerminals	3/10	70%	0.02869	0.02982	0.01475	0.01455	0.1033	0.1035	0.01109	0.00606	2.40 ×
sumSubtreeHints	3/10	70%	0.02907	0.02895	0.01643	0.01495	0.1028	0.103	0.01059	0.006059	2.47 ×
autocompleteTopKProxy	5/10	50%	0.02888	0.0307	0.01799	0.01557	0.1087	0.1088	0.01306	0.009512	1.64 ×
countLazyNodes	4/10	60%	0.02927	0.03051	0.01645	0.01508	0.1027	0.1028	0.01021	0.006851	2.20 ×

Table 60: What each optimization bought, per fold pass, for **Trie** (percent; companion to Table 59). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘—’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
sumPrefixFreq	+99.16%	−0.7162%	+1133%	+40.98%
countTerminals	+94.47%	+1.354%	+831.9%	+82.99%
sumSubtreeHints	+76.95%	+9.844%	+871.5%	+74.72%
autocompleteTopKProxy	+60.53%	+15.56%	+732.3%	+37.29%
countLazyNodes	+77.87%	+9.133%	+905.1%	+49.07%

Table 61: Per-pass map performance for **Trie**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS										SoA										$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$	$S_{\ell bv}^{+av}$	
decayTrieStats	6/9	67%	0.08269	0.0832	0.08128	0.0778	0.07834	0.07329	0.07288	0.2505	0.2508	0.09385	0.09139	0.09498	0.09192	0.09044	0.02635	0.02601	0.02682	0.02593	2.81 $\times$
resetTraversalState	7/9	78%	0.08237	0.08022	0.07635	0.07934	0.08144	0.07095	0.07209	0.2465	0.2452	0.09244	0.08934	0.09136	0.09078	0.08835	0.01596	0.01623	0.01652	0.01626	4.44 $\times$

Table 62: What each optimization bought, per map pass, for **Trie** (percent; companion to Table 61). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘—’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA									
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$	
decayTrieStats	+1.736%	+3.75%	-0.6804%	+6.163%	+0.5612%	+166.9%	-1.197%	-3.789%	-0.5853%	+1.644%	+248.9%	+1.3%	-1.764%	+3.436%	
resetTraversalState	+7.893%	-6.26%	-2.586%	+11.82%	-1.579%	+166.7%	+1.185%	-2.204%	-1.58%	+2.75%	+468.6%	-1.645%	-3.348%	+1.587%	

Table 63: Per-pass fold performance for **ColorOmtree**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
paletteEntriesQuantized	13/25	48%	0.05031	0.05047	0.0414	0.0415	-	-	0.03606	0.03657	$1.15\times$
quantizationErrorProxy	10/25	60%	0.06321	0.06345	0.04586	0.04643	-	-	0.04682	0.04898	$0.98\times$

Table 64: What each optimization bought, per fold pass, for **ColorOmtree** (percent; companion to Table 63). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘—’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
paletteEntriesQuantized	+21.52%	−0.2434%	−	−1.386%
quantizationErrorProxy	+37.83%	−1.234%	−	−4.426%

Table 65: Per-pass map performance for **Add1TreeInt8**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS									SoA										$A^{\min}/S^{\min}$	
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$		$S_{\ell bv}^{+av}$
add1Tree	1/2	50%	0.04423	0.04554	0.035	0.03762	0.03891	0.02056	0.02259	0.1038	0.1039	0.03333	0.03542	0.03479	0.01034	0.006933	0.003739	0.00215	0.002144	0.002102	9.78×

Table 66: What each optimization bought, per map pass, for **Add1TreeInt8** (percent; companion to Table 65). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
add1Tree	+26.38%	−10.06%	−3.328%	+82.91%	−8.96%	+211.5%	−4.2%	+1.814%	+242.5%	+49.16%	+176.6%	+73.91%	+74.39%	+1.998%

Table 67: Per-pass map performance for **Add1TreeInt16**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS										SoA										$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$	$S_{\ell bv}^{+av}$	
add1Tree	1/2	50%	0.04625	0.04765	0.03427	0.03484	0.03549	0.02235	0.02395	0.1056	0.1057	0.03572	0.037	0.0365	0.01232	0.0102	0.005396	0.004535	0.004735	0.004309	5.19×

Table 68: What each optimization bought, per map pass, for **Add1TreeInt16** (percent; companion to Table 67). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
add1Tree	+34.97%	−3.454%	−1.845%	+55.88%	−6.684%	+195.5%	−2.137%	+1.384%	+200.4%	+20.79%	+128.2%	+18.99%	+13.96%	+9.886%

Table 69: Per-pass map performance for **Add1TreeInt32**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS									SoA										$A^{\min}/S^{\min}$	
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$		$S_{\ell bv}^{+av}$
add1Tree	1/2	50%	0.05168	0.05183	0.03821	0.0394	0.04277	0.0268	0.0259	0.1118	0.1113	0.04028	0.04155	0.04084	0.01849	0.01448	0.01017	0.00898	0.009359	0.009247	2.88×

Table 70: What each optimization bought, per map pass, for **Add1TreeInt32** (percent; companion to Table 69). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
add1Tree	+35.25%	−10.66%	−7.873%	+47.01%	+3.487%	+177.6%	−1.376%	+1.721%	+124.7%	+27.75%	+81.88%	+13.22%	+8.633%	+1.211%

Table 71: Per-pass map performance for **Add1TreeInt64**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS									SoA										$A^{\min}/S^{\min}$	
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$		$S_{\ell bv}^{+av}$
add1Tree	1/2	50%	0.06123	0.06102	0.04783	0.05106	0.04768	0.03667	0.03623	0.121	0.1229	0.04995	0.04922	0.05029	0.02832	0.02399	0.01906	0.01855	0.01832	0.02025	1.98×

Table 72: What each optimization bought, per map pass, for **Add1TreeInt64** (percent; companion to Table 71). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_{ℓ}^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_{ℓ}^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
add1Tree	+28.02%	+0.3209%	+7.096%	+39.26%	+1.195%	+142.3%	-0.6939%	-2.129%	+73.78%	+18.09%	+48.58%	+2.771%	+4.09%	-9.547%

Table 73: Per-pass map performance for **ArithmeticIntensityInt8**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS									SoA										$A^{\min}/S^{\min}$	
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$		$S_{\ell bv}^{+av}$
arithKernel	1/2	50%	0.4023	0.4038	0.3831	0.4942	0.3797	0.4854	0.3675	0.5661	0.5649	0.3879	0.4862	0.3884	0.4545	0.08455	0.4368	0.07218	0.07488	0.07446	5.09×

Table 74: What each optimization bought, per map pass, for **ArithmeticIntensityInt8** (percent; companion to Table 73). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_{ℓ}^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_{ℓ}^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
arithKernel	+4.997%	+0.91%	+30.16%	+1.813%	+32.08%	+45.93%	-0.1226%	+25.19%	+6.979%	+437.6%	+4.049%	+505.2%	+483.4%	+0.5627%

Table 75: Per-pass map performance for **ArithmeticIntensityInt16**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS									SoA										$A^{\min}/S^{\min}$	
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell bv}^{-av}$		$S_{\ell bv}^{+av}$
arithKernel	1/2	50%	0.2068	0.2069	0.1874	0.4881	0.1827	0.4955	0.1659	0.4146	0.4144	0.1955	0.4951	0.1854	0.4645	0.0675	0.4554	0.05501	0.05573	0.05497	3.02×

Table 76: What each optimization bought, per map pass, for **ArithmeticIntensityInt16** (percent; companion to Table 75). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_{ℓ}^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_{ℓ}^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell bv}^S$
arithKernel	+10.36%	+2.6%	+167.2%	-1.479%	+198.7%	+112.1%	+5.4%	+167%	+6.575%	+588.2%	+2%	+727.9%	+717.1%	+1.395%

Table 77: Per-pass map performance for **ArithmeticIntensityInt32**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS										SoA										$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	S_{lb}^{-av}	S_{lb}^{+av}	$S_{\ell bv}^{-av}$	$S_{\ell bv}^{+av}$	
arithKernel	1/2	50%	0.3699	0.3691	0.3453	0.5107	0.3385	0.5028	0.3197	0.5148	0.5146	0.3488	0.5044	0.3372	0.4694	0.1545	0.4531	0.1452	0.1407	0.1395	2.29 $\times$

Table 78: What each optimization bought, per map pass, for **ArithmeticIntensityInt32** (percent; companion to Table 77). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA								
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av lb}^S$	Δ_v^S	$\Delta_{av lbv}^S$
arithKernel	+7.144%	+1.987%	+50.84%	+1.557%	+57.27%	+47.58%	+3.455%	+49.6%	+7.46%	+203.9%	+3.587%	+212%	+222%	+0.8599%

Table 79: Per-pass map performance for **ArithmeticIntensityInt64**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS										SoA										$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	S_{lb}^{-av}	S_{lb}^{+av}	$S_{\ell bv}^{-av}$	$S_{\ell bv}^{+av}$	
arithKernel	1/2	50%	0.5365	0.5354	0.5076	0.4969	0.4995	0.4987	0.5015	0.5866	0.5871	0.4962	0.4992	0.4973	0.4916	0.4856	0.4718	0.4724	0.4711	0.4712	1.05 $\times$

Table 80: What each optimization bought, per map pass, for **ArithmeticIntensityInt64** (percent; companion to Table 79). Every column is $(baseline - feature\ enabled) / feature\ enabled$, which is identically $(speedup - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘-’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA									
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av lb}^S$	Δ_v^S	$\Delta_{av lbv}^S$	
arithKernel	+5.69%	+1.621%	-0.5183%	-0.3547%	-0.5553%	+18.22%	-0.2268%	+0.3774%	+1.549%	+1.218%	+4.182%	-0.127%	+0.1513%	-0.01337%	

Table 81: Per-pass fold performance for **PiecewiseFunctions**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (**-mavx2**); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. **Uses** is the fields this pass accesses out of the benchmark ADT’s total (recursive and non-recursive alike), and **Dead%** the fraction it never touches – the quantity a struct-of-arrays layout exists to exploit, since an unused field costs an AoS traversal bandwidth it cannot avoid. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns are the recursive configurations only: the loopified pair Table 1 contrasts is absent here, because nothing in a fold is loopifiable. The two tables therefore compare different binaries of the same source, and where a program’s two layouts sit within a few percent of each other that is enough to reverse which one leads. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass			AoS				SoA				$A^{\min}/S^{\min}$
	Uses	Dead%	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{+av}	
norm2Estimate	5/8	38%	0.06382	0.06376	0.0265	0.0272	0.3296	0.3137	0.03883	0.0227	1.17×
truncateTolViolations	3/8	62%	0.04876	0.04839	0.0261	0.02385	0.2741	0.2647	0.01847	0.01183	2.02×
compressMass	3/8	62%	0.04747	0.04852	0.02694	0.02509	0.2812	0.2831	0.01484	0.01192	2.10×
autorefineMaxLevel	4/8	50%	0.04765	0.04747	0.02765	0.02745	0.293	0.2903	0.02103	0.02033	1.35×
pmapCutHistogram	4/8	50%	0.05935	0.05973	0.02715	0.02858	0.2806	0.2823	0.02015	0.01368	1.98×
lbDeuxLoadProxy	5/8	38%	0.04815	0.0484	0.02587	0.02593	0.3031	0.2913	0.02148	0.01797	1.44×

Table 82: What each optimization bought, per fold pass, for **PiecewiseFunctions** (percent; companion to Table 81). Every column is $(\text{baseline} - \text{feature enabled}) / \text{feature enabled}$, which is identically $(\text{speedup} - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘–’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS		SoA	
	Δ_m^A	Δ_t^A	Δ_m^S	Δ_t^S
norm2Estimate	+140.8%	−2.552%	+748.7%	+71.07%
truncateTolViolations	+86.79%	+9.46%	+1384%	+56.13%
compressMass	+76.24%	+7.366%	+1796%	+24.43%
autorefineMaxLevel	+72.29%	+0.7542%	+1293%	+3.418%
pmapCutHistogram	+118.6%	−5.007%	+1293%	+47.24%
lbDeuxLoadProxy	+86.13%	−0.2545%	+1311%	+19.51%

Table 83: Per-pass map performance for **PiecewiseFunctions**. Times are median per iteration (s), 4 significant digits. Columns are compiled configurations; see Table 3 for the symbol key. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. In each row the fastest configuration is **green** and the slowest **red**. Σ_b is the BUFFERS this pass leaves unmodified, out of the fully factored value’s total, and $\Sigma_b\%$ that as a fraction. It is read from the pass’s own source annotation – a property of what the pass WRITES, and therefore an upper bound on what selective buffer sharing (b) could exploit, NOT a measurement of what the optimization achieved. Whether sharing fired is the Δ_b^S column of the companion delta table: a pass can read Σ_b near 100% and Δ_b^S near zero, which is what a traversal that declined loopification looks like – sharing is implemented on the loopified form, so a pass that is not loopified is not shared either. A factored value is one buffer per scalar field plus the constructor stream, and a dependence-free map shares that constructor stream too – it rebuilds the same shape, so the tags are copied unchanged. Counting fields alone would understate every map by one buffer. A map copies every field into the output region, so ”fields used” is vacuously all of them and distinguishes nothing; what separates one map from another is how much of the data it merely COPIES. Recursive child fields are not buffers and are excluded from both counts. $A^{\min}/S^{\min}$ divides that row’s fastest AoS configuration by its fastest SoA one, so each layout is represented by whichever configuration actually served this pass best; $>1\times$ means SoA is faster. The minimum is taken over the columns THIS table shows and no others, so the ratio can be checked against the row above it. Those columns include the pair Table 1 contrasts, but that table reports a pass-SUM per program while each row here is one pass, so a program with several map passes can lead here and trail there. A cell with no time names its failure: ‘*’ did not compile, ‘-’ compiled but the executable failed to run, ‘**’ ran but its output did not match the oracle (an empty output included), ‘‡’ exhausted the C stack, and ‘?’ was not measured – either not run in this campaign or having no registered oracle to check it against. A ‘‡’ is a RESULT, not a defect: that configuration recurses once per element where mutable cursors and tail calls produce a loop, and the curated inputs are sized for the loop. The benchmark driver prints a warning naming each one.

Pass	AoS										SoA										$A^{\min}/S^{\min}$
	Σ_b	$\Sigma_b\%$	A_{ri}^{-t}	A_{ri}	A_{rm}^{-t}	A_{rm}^{-av}	A_{rm}^{+av}	A_{ℓ}^{-av}	A_{ℓ}^{+av}	S_{ri}^{-t}	S_{ri}	S_{rm}^{-t}	S_{rm}^{-av}	S_{rm}^{+av}	S_{ℓ}^{-av}	S_{ℓ}^{+av}	$S_{\ell b}^{-av}$	$S_{\ell b}^{+av}$	$S_{\ell b v}^{-av}$	$S_{\ell b v}^{+av}$	
addConstPW	6/7	86%	0.1307	0.1316	0.1245	0.1262	0.1237	0.1088	0.1085	0.5134	0.5051	0.1411	0.1399	0.1346	0.1044	0.09841	0.01615	0.01542	0.01541	0.01552	7.04 ×
diffPW	5/7	71%	0.1305	0.1296	0.1259	0.1296	0.1274	0.1163	0.1144	0.5405	0.5412	0.146	0.1393	0.1397	0.1123	0.1072	0.0365	0.0346	0.03534	0.03477	3.31 ×

Table 84: What each optimization bought, per map pass, for **PiecewiseFunctions** (percent; companion to Table 83). Every column is $(\text{baseline} - \text{feature enabled}) / \text{feature enabled}$, which is identically $(\text{speedup} - 1) \times 100$: +100% means the feature made the pass twice as fast, +700% eight times as fast, and a negative value means it made the pass slower. Percentages are scale-free, so they are comparable across passes whose absolute times differ widely. See Table 4 for each column’s exact definition. ‘–’ marks a pass where either side of the difference was not measured; the timing table above says which.

Pass	AoS					SoA									
	Δ_m^A	Δ_t^A	$\Delta_{av rm}^A$	Δ_ℓ^A	$\Delta_{av \ell}^A$	Δ_m^S	Δ_t^S	$\Delta_{av rm}^S$	Δ_ℓ^S	$\Delta_{av \ell}^S$	Δ_b^S	$\Delta_{av \ell b}^S$	Δ_v^S	$\Delta_{av \ell b v}^S$	
addConstPW	+4.953%	+0.6858%	+2.06%	+16.01%	+0.2738%	+263.9%	+4.851%	+3.973%	+34.02%	+6.071%	+546.2%	+4.746%	+4.828%	−0.6831%	
diffPW	+3.61%	−1.165%	+1.746%	+11.43%	+1.644%	+270.3%	+4.513%	−0.2456%	+24.03%	+4.795%	+207.8%	+5.494%	+3.291%	+1.642%	

Table 85: Integer-width high-arithmetic-intensity vectorization. Every configuration was compiled for the same SIMD target, 256-bit AVX2 (`-mavx2`); Gibbon’s vectorizer and the C compiler’s auto-vectorizer both target it, so the columns are like-for-like. Every column is compiled with the C compiler’s auto-vectorizer disabled (`--no-gcc-vectorize`, which suppresses basic-block/SLP vectorization as well as loop vectorization), so the SIMD column isolates Gibbon’s own vectorizer rather than the backend’s; absolute times are correspondingly higher than a default build. Selective buffer sharing is enabled from the **SoA loopify shared** column onwards, including the SIMD column, so those cells are directly comparable with the per-program map tables, whose SoA columns also enable it. **SIMD spd.** divides the shared column by the SIMD one – the two differ by the vectorizer alone, so it reports what vectorization buys and nothing else; **AoS-vs-SIMD** keeps AoS as its baseline and therefore includes every SoA-side optimization. Int64’s packed multiply is EMULATED, and its SIMD column is compiled with `--simd-w64-multiply` to measure it. x86 has no packed 64-bit multiply below AVX-512DQ, so the helper synthesizes one from three packed $32\times 32\rightarrow 64$ multiplies plus two shifts and two adds, all in registers – correct SIMD, but seven instructions for two lanes against one `imul` per lane scalar. The compiler therefore refuses it by default; the number here is what enabling it costs, and a speedup below $1\times$ in that row is the expected result rather than a defect.

Width	Ops/elem	Bytes ld/elem	Bytes st/elem	Ops/byte	AoS raw	SoA raw	SoA loopify	SoA loopify shared	SoA SIMD	SIMD spd.	AoS-vs-SIMD	Status	Qual.
Int8	415	1	1	207.50	0.491200	0.485164	0.452644	0.435130	0.074425	5.85 ×	6.60 ×	SIMD	OK
Int16	415	2	2	103.75	0.494474	0.495715	0.463455	0.443148	0.054269	8.17 ×	9.11 ×	SIMD	OK
Int32	415	4	4	51.88	0.510793	0.508114	0.469934	0.451894	0.139847	3.23 ×	3.65 ×	SIMD	OK
Int64	415	8	8	25.94	0.497270	0.499846	0.493153	0.475412	0.547004	0.87×	0.91×	SIMD (emulated multiply)	OK